第六章作业：基于 RNN 的古诗生成

姓名：王博　学号：2351563

2026 年 3 月 30 日

目录

1 RNN、LSTM、GRU 模型解释 **2**
1.1 循环神经网络（RNN） 2
1.1.1 基本结构 2
1.1.2 优缺点 2
1.2 长短期记忆网络（LSTM） 2
1.2.1 核心组件 2
1.2.2 优势 3
1.3 门控循环单元（GRU） 3
1.3.1 核心公式 3
1.3.2 与 LSTM 的比较 3

2 诗歌生成过程叙述 **4**
2.1 数据预处理 4
2.2 模型构建 4
2.3 模型训练 5
2.4 诗歌生成 5

3 诗歌生成结果 **6**
3.1 训练过程截图 6
3.2 诗歌生成结果 6
3.3 生成结果截图 7

4 实验总结 **7**
4.1 实验收获 7
4.2 存在的问题与改进方向 7

1 RNN、LSTM、GRU 模型解释

1.1 循环神经网络（RNN）

循环神经网络（Recurrent Neural Network, RNN）是一类专门用于处理**序列数据**的神经网络。与前馈神经网络不同，RNN 在隐藏层中引入了循环连接，使网络能够保留对先前输入的”记忆”。

1.1.1 基本结构

RNN 的核心思想是在时间步 t 处，隐藏状态 h_t 不仅取决于当前输入 x_t，还取决于上一时间步的隐藏状态 h_{t-1}。其前向传播公式为：

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{1}$$

$$y_t = W_{hy}h_t + b_y \tag{2}$$

其中，W_{xh}、W_{hh}、W_{hy} 分别为输入到隐藏层、隐藏层到隐藏层、隐藏层到输出层的权重矩阵，b_h 和 b_y 为偏置项。

1.1.2 优缺点

- **优点**：结构简单，能够处理变长序列，参数共享。
- **缺点**：存在严重的**梯度消失**和**梯度爆炸**问题，难以捕获长距离依赖关系。当序列较长时，早期的信息会在反复乘以权重矩阵的过程中逐渐衰减或爆炸。

1.2 长短期记忆网络（LSTM）

长短期记忆网络（Long Short-Term Memory, LSTM）是为了解决标准 RNN 的梯度消失问题而设计的变体。它通过引入**门控机制**（Gate Mechanism）和**细胞状态**（Cell State），有效地控制信息的流动。

1.2.1 核心组件

LSTM 包含三个门和一个细胞状态：

1. **遗忘门（Forget Gate）**：决定从细胞状态中丢弃哪些信息：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3}$$

2. **输入门（Input Gate）**：决定哪些新信息被加入到细胞状态中：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{4}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{5}$$

3. **细胞状态更新**：

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{6}$$

4. **输出门（Output Gate）**：决定输出哪些信息：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{7}$$

$$h_t = o_t \odot \tanh(C_t) \tag{8}$$

其中 σ 为 Sigmoid 函数，$\odot$ 为逐元素相乘（Hadamard 积）。

1.2.2 优势

LSTM 通过细胞状态 C_t 实现了信息的长距离传递——梯度可以沿着细胞状态几乎无损地向后传播，从而有效缓解了梯度消失问题。

1.3 门控循环单元（GRU）

门控循环单元（Gated Recurrent Unit, GRU）是 LSTM 的一种简化变体，由 Cho 等人于 2014 年提出。GRU 将 LSTM 的遗忘门和输入门合并为一个**更新门**（Update Gate），同时合并了细胞状态和隐藏状态。

1.3.1 核心公式

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad \text{（更新门）} \tag{9}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad \text{（重置门）} \tag{10}$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad \text{（候选隐藏状态）} \tag{11}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{（最终隐藏状态）} \tag{12}$$

1.3.2 与 LSTM 的比较

表 1: LSTM 与 GRU 的对比

特性	**LSTM**	**GRU**
门的数量	3 个（遗忘、输入、输出）	2 个（更新、重置）
状态数量	2 个（h_t 和 C_t）	1 个（h_t）
参数数量	较多	较少
训练速度	较慢	较快
长序列表现	优秀	良好

GRU 在参数更少的情况下通常能达到与 LSTM 接近的性能，适合数据量较小或计算资源有限的场景。

2 诗歌生成过程叙述

本实验基于 PyTorch 框架，使用 LSTM 网络实现中文古诗的自动生成。整个过程可分为**数据预处理**、**模型构建**、**模型训练**和**诗歌生成**四个阶段。

2.1 数据预处理

1. **读取语料**：从 `poems.txt` 文件中逐行读取古诗，每行格式为" 标题: 内容"。过滤掉包含特殊字符或长度不合适的诗句。
2. **添加标记**：在每首诗前后分别添加起始标记 `G` 和结束标记 `E`，用于标记诗歌的起止。
3. **构建词表**:统计所有字的出现频率,按频率降序排列,建立字到索引的映射 `word_int_map` 以及索引到字的映射 `vocabularies`。
4. **向量化**：将每首诗转换为对应的整数索引序列，得到 `poems_vector`。
5. **批量生成**:按 `batch_size` 将数据分成多个批次。对于每个输入序列 $x = [w_1, w_2, \ldots, w_n]$，对应的标签为 $y = [w_2, w_3, \ldots, w_n, w_n]$，即将序列右移一位。

2.2 模型构建

模型采用 **Embedding + LSTM + Dense** 的经典结构：

1. **词嵌入层**（`word_embedding`）：将离散的字索引映射为 100 维的稠密向量。权重使用均匀分布 $U(-1, 1)$ 随机初始化。
2. **LSTM 层**（`rnn_lstm`）：核心序列建模模块，配置如下：
   - 输入维度：100（与词嵌入维度一致）
   - 隐藏维度：128
   - 层数：2 层（堆叠 LSTM）
   - `batch_first=True`

   LSTM 的初始隐藏状态 h_0 和细胞状态 c_0 均初始化为零向量。
3. **全连接层**（`fc`）：将 LSTM 的 128 维输出映射到词表大小的空间，再经过 ReLU 激活和 LogSoftmax 得到每个字的对数概率分布。

其关键补全代码为:

```
# 定义 LSTM
self.rnn_lstm = nn.LSTM(
    input_size=embedding_dim,
    hidden_size=lstm_hidden_dim,
    num_layers=2,
    batch_first=True
)

# 前向传播中调用 LSTM
h_0 = torch.zeros(2, batch_input.size(0), self.lstm_dim)
c_0 = torch.zeros(2, batch_input.size(0), self.lstm_dim)
output, (h_n, c_n) = self.rnn_lstm(batch_input, (h_0, c_0))
```

Listing 1: LSTM 模型定义

2.3 模型训练

1. **损失函数**: 采用负对数似然损失（NLLLoss），与 LogSoftmax 配合使用，等价于交叉熵损失。
2. **优化器**: 使用 RMSprop 优化器，学习率为 0.01。
3. **训练过程**: 共训练 30 个 epoch。每个 epoch 中，遍历所有 batch，对 batch 内的每条诗歌逐条计算损失后取平均，执行反向传播和梯度更新。使用梯度裁剪（`clip_grad_norm_`，阈值为 1）防止梯度爆炸。
4. **模型保存**: 每隔 20 个 batch 保存一次模型参数。

2.4 诗歌生成

生成阶段使用训练好的模型，以指定的开头字逐字生成诗歌:

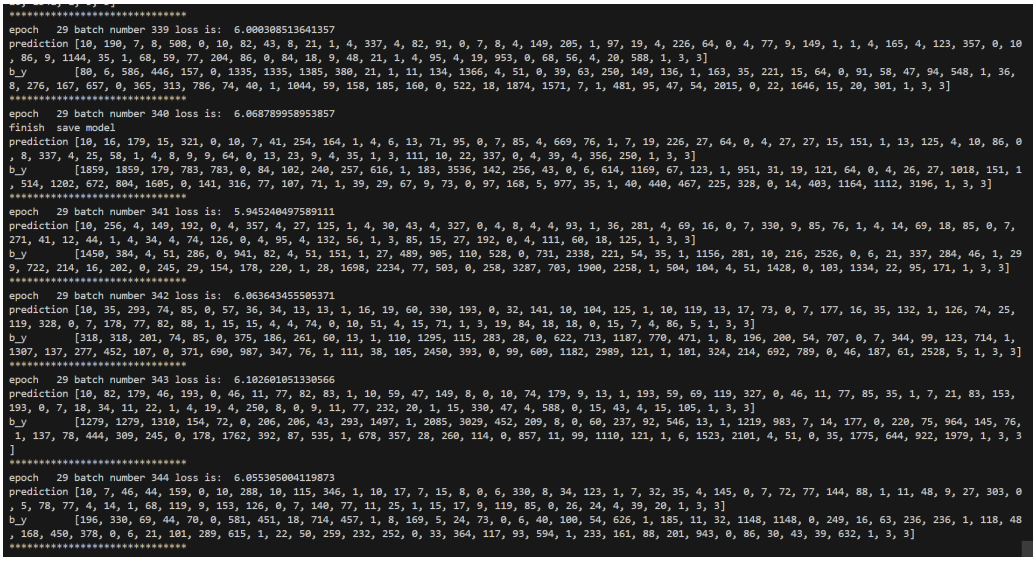
1. 将开头字（如”日”）作为初始输入。
2. 将当前已生成的所有字转换为索引序列，送入模型。
3. 模型输出最后一个时间步的概率分布，取概率最大的字作为下一个生成字。
4. 将新生成的字追加到已有序列中，重复步骤 2-3。
5. 当生成结束标记 E 或序列长度超过 30 时停止。

该过程本质上是一个**自回归**（Autoregressive）的解码过程——每一步的输出都作为下一步的输入，直到满足终止条件。

3 诗歌生成结果

以下展示了以”日、红、山、夜、湖、海、月”作为开头字，使用训练好的 LSTM 模型生成的诗歌结果。

3.1 训练过程截图



```
******************************
epoch   29 batch number 339 loss is:  6.000308513641357
prediction [10, 190, 7, 8, 508, 0, 10, 82, 43, 8, 21, 1, 4, 337, 4, 82, 91, 0, 7, 8, 4, 149, 205, 1, 97, 19, 4, 226, 64, 0, 4, 77, 9, 149, 1, 1, 4, 165, 4, 123, 357, 0, 10
, 86, 9, 1144, 35, 1, 68, 59, 77, 204, 86, 0, 84, 18, 9, 48, 21, 1, 4, 95, 4, 19, 953, 0, 68, 56, 4, 20, 588, 1, 3, 3]
b_y       [80, 6, 586, 446, 157, 0, 1335, 1335, 1385, 380, 21, 1, 11, 134, 1366, 4, 51, 0, 39, 63, 250, 149, 136, 1, 163, 35, 221, 15, 64, 0, 91, 58, 47, 94, 548, 1, 36,
8, 276, 167, 657, 0, 365, 313, 786, 74, 40, 1, 1044, 59, 158, 185, 160, 0, 522, 18, 1874, 1571, 7, 1, 481, 95, 47, 54, 2015, 0, 22, 1646, 15, 20, 301, 1, 3, 3]
******************************
epoch   29 batch number 340 loss is:  6.068789958953857
finish  save model
prediction [10, 16, 179, 15, 321, 0, 10, 7, 41, 254, 164, 1, 4, 6, 13, 71, 95, 0, 7, 85, 4, 669, 76, 1, 7, 19, 226, 27, 64, 0, 4, 27, 27, 15, 151, 1, 13, 125, 4, 10, 86, 0
, 8, 337, 4, 25, 58, 1, 4, 8, 9, 9, 64, 0, 13, 23, 9, 4, 35, 1, 3, 111, 10, 22, 337, 0, 4, 39, 4, 356, 250, 1, 3, 3]
b_y       [1859, 1859, 179, 783, 783, 0, 84, 102, 240, 257, 616, 1, 183, 3536, 142, 256, 43, 0, 6, 614, 1169, 67, 123, 1, 951, 31, 19, 121, 64, 0, 4, 26, 27, 1018, 151, 1
, 514, 1202, 672, 804, 1605, 0, 141, 316, 77, 107, 71, 1, 39, 29, 67, 9, 73, 0, 97, 168, 5, 977, 35, 1, 40, 440, 467, 225, 328, 0, 14, 403, 1164, 1112, 3196, 1, 3, 3]
******************************
epoch   29 batch number 341 loss is:  5.945240497589111
prediction [10, 256, 4, 149, 192, 0, 4, 357, 4, 27, 125, 1, 4, 30, 43, 4, 327, 0, 4, 8, 4, 4, 93, 1, 36, 281, 4, 69, 16, 0, 7, 330, 9, 85, 76, 1, 4, 14, 69, 18, 85, 0, 7,
271, 41, 12, 44, 1, 4, 34, 4, 74, 126, 0, 4, 95, 4, 132, 56, 1, 3, 85, 15, 27, 192, 0, 4, 111, 60, 18, 125, 1, 3, 3]
b_y       [1450, 384, 4, 51, 286, 0, 941, 82, 4, 51, 151, 1, 27, 489, 905, 110, 528, 0, 731, 2338, 221, 54, 35, 1, 1156, 281, 10, 216, 2526, 0, 6, 21, 337, 284, 46, 1, 29
9, 722, 214, 16, 202, 0, 245, 29, 154, 178, 220, 1, 28, 1698, 2234, 77, 503, 0, 258, 3287, 703, 1900, 2258, 1, 504, 104, 4, 51, 1428, 0, 103, 1334, 22, 95, 171, 1, 3, 3]
******************************
epoch   29 batch number 342 loss is:  6.0636434555505371
prediction [10, 35, 293, 74, 85, 0, 57, 36, 34, 13, 13, 1, 16, 19, 60, 330, 193, 0, 32, 141, 10, 104, 125, 1, 10, 119, 13, 17, 73, 0, 7, 177, 16, 35, 132, 1, 126, 74, 25,
119, 328, 0, 7, 178, 77, 82, 88, 1, 15, 15, 4, 4, 74, 0, 10, 51, 4, 15, 71, 1, 3, 19, 84, 18, 18, 0, 15, 7, 4, 86, 5, 1, 3, 3]
b_y       [318, 318, 201, 74, 85, 0, 375, 186, 261, 60, 13, 1, 110, 1295, 115, 283, 28, 0, 622, 713, 1187, 770, 471, 1, 8, 196, 200, 54, 707, 0, 7, 344, 99, 123, 714, 1,
1307, 137, 277, 452, 107, 0, 371, 690, 987, 347, 76, 1, 111, 38, 105, 2450, 393, 0, 99, 609, 1182, 2989, 121, 1, 101, 324, 214, 692, 789, 0, 46, 187, 61, 2528, 5, 1, 3, 3]
******************************
epoch   29 batch number 343 loss is:  6.1026010513305 66
prediction [10, 82, 179, 46, 193, 0, 46, 11, 77, 82, 83, 1, 10, 59, 47, 149, 8, 0, 10, 74, 179, 9, 13, 1, 193, 59, 69, 119, 327, 0, 46, 11, 77, 85, 35, 1, 7, 21, 83, 153,
193, 0, 7, 18, 34, 11, 22, 1, 4, 19, 4, 250, 8, 0, 9, 11, 77, 232, 20, 1, 15, 330, 47, 4, 588, 0, 15, 43, 4, 15, 105, 1, 3, 3]
b_y       [1279, 1279, 1310, 154, 72, 0, 206, 206, 43, 293, 1497, 1, 2085, 3029, 452, 209, 8, 0, 60, 237, 92, 546, 13, 1, 1219, 983, 7, 14, 177, 0, 220, 75, 964, 145, 76,
 1, 137, 78, 444, 309, 245, 0, 178, 1762, 392, 87, 535, 1, 678, 357, 28, 260, 114, 0, 857, 11, 99, 1110, 121, 1, 6, 1523, 2101, 4, 51, 0, 35, 1775, 644, 922, 1979, 1, 3, 3
]
******************************
epoch   29 batch number 344 loss is:  6.055305004119873
prediction [10, 7, 46, 44, 159, 0, 10, 288, 10, 115, 346, 1, 10, 17, 7, 15, 8, 0, 6, 330, 8, 34, 123, 1, 7, 32, 35, 4, 145, 0, 7, 72, 77, 144, 88, 1, 11, 48, 9, 27, 303, 0
, 5, 78, 77, 4, 14, 1, 68, 119, 9, 153, 126, 0, 7, 140, 77, 11, 25, 1, 15, 17, 9, 119, 85, 0, 26, 24, 4, 39, 20, 1, 3, 3]
b_y       [196, 330, 69, 44, 70, 0, 581, 451, 18, 714, 457, 1, 8, 169, 5, 24, 73, 0, 6, 40, 100, 54, 626, 1, 185, 11, 32, 1148, 1148, 0, 249, 16, 63, 236, 236, 1, 118, 48
, 168, 450, 378, 0, 6, 21, 101, 289, 615, 1, 22, 50, 259, 232, 252, 0, 33, 364, 117, 93, 594, 1, 233, 161, 88, 201, 943, 0, 86, 30, 43, 39, 632, 1, 3, 3]
******************************
```

图 1: LSTM 模型训练过程截图

3.2 诗歌生成结果

使用以下 7 个汉字分别作为开头字（begin word），由训练好的 LSTM 模型生成诗歌，结果如下：

表 2: 以指定开头字生成的诗歌

开头字	生成诗歌
日	日云浮树，风光生九州。不知何处客，不见此中人。何事蓬山老，何年
红	红玉风。上国无人知，今朝有一言。何当一何在，时复一相思
山	山古树，风起春光满。若教无子知难，不得中生作老
夜	夜悠悠，风起春风起，夜色不可见，一枝无处春不知何处在，不见此
湖	胡水上天。不知何处，乐以以为。
海	海草年尘。风光有花气，不见此心中。何事不可见，一枝无处春
月	月，悠悠，风流不可寻。不知何处，万国在天。

3.3 生成结果截图

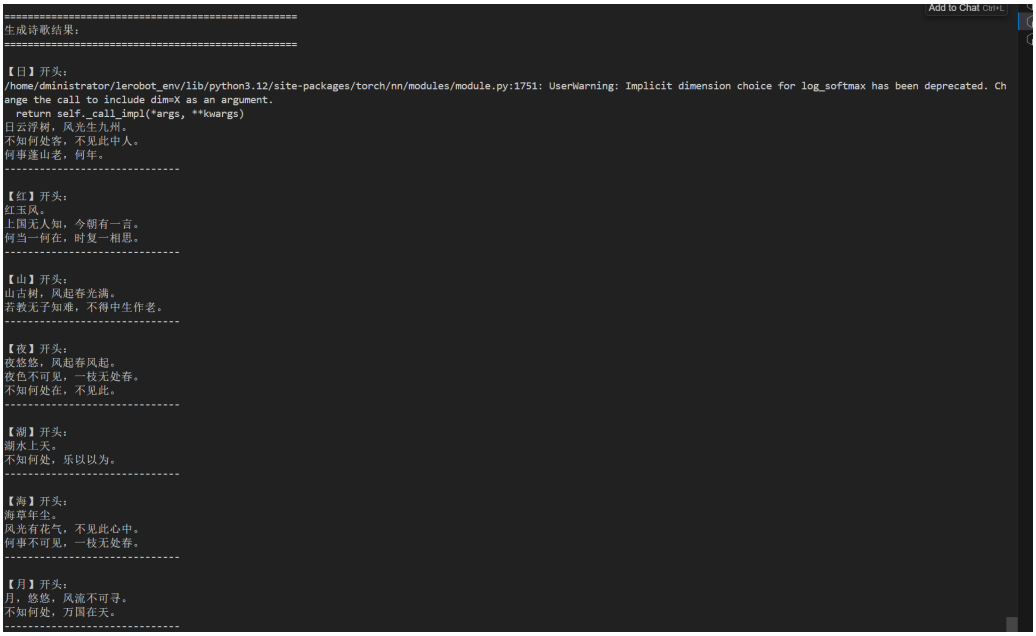


图 2: 诗歌生成终端运行截图

4 实验总结

4.1 实验收获

1. **深入理解了 RNN 系列模型**：通过本次实验，对 RNN、LSTM、GRU 三种循环神经网络的结构原理有了更加深入的认识。特别是 LSTM 通过门控机制解决长距离依赖问题的思想，在实际编码中得到了验证。

2. **掌握了序列生成的建模方法**：理解了语言模型的本质——通过学习 $P(w_t|w_1, w_2, \ldots, w_{t-1})$ 的条件概率分布，实现逐字符的自回归生成。训练时使用 Teacher Forcing 策略，生成时使用贪心解码。

3. **PyTorch 实践能力提升**：通过补全 LSTM 的定义和前向传播代码，加深了对 PyTorch 中 `nn.LSTM` 接口的理解，包括隐藏状态的初始化、输入输出维度的对齐等。

4.2 存在的问题与改进方向

1. **生成质量**：当前模型生成的诗歌在语义连贯性和格律方面仍有不足，可以通过增大模型容量、使用更大的数据集或引入注意力机制来改善。

2. **解码策略**：实验中使用了贪心解码（取概率最大的字），可以尝试**采样解码**（按概率分布采样）或 **Beam Search** 等策略，以增加生成诗歌的多样性。

3. **格律约束**：古诗对平仄、押韵有严格要求，可以在生成过程中加入格律规则作为额外约束，提升生成诗歌的形式美感。

参考文献

[1] Xingxing Zhang and Mirella Lapata. 2014. Chinese poetry generation with recurrent neural networks. In *Proceedings of the 2014 Conference on EMNLP*. Association for Computational Linguistics, October.

[2] Hochreiter S, Schmidhuber J. Long short-term memory[J]. *Neural Computation*, 1997, 9(8): 1735-1780.

[3] Cho K, van Merrienboer B, Gulcehre C, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation[C]. *EMNLP*, 2014.